Rings: An Algebraic Structured Peer-to-Peer Network for Decentralized Identity and Resource Ownership

Ryan J. Kung

Abstract

Rings studies an implemented structured peer-to-peer network as an algebraic object for decentralized identity and resource ownership. Its motivating failure mode is authority collapse:

$$\mathsf{root}(id, data, route) \in \{\mathsf{server}, \mathsf{dns}, \mathsf{relay}, \mathsf{platform}\}.$$

The paper does not claim a new DHT, DID method, WebRTC transport, or CRDT merge law. Its claim is that these mechanisms can be composed through a carrier-separated ownership model:

$$\mathsf{Rings} = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathsf{T}_A^{\mathcal{E}}, \Pi, \mathbb{G}, \mathcal{X})$$

with

$$\begin{aligned} R &= \mathbb{Z}/2^{160}\mathbb{Z}, \\ \mathcal{R}_N &= \text{Correct-Chord overlay object}, \\ \Omega_N(X,H) &= (X, \mathrm{owner}_N \circ H), \\ \tau_\nu &: S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu). \end{aligned}$$

$$\begin{aligned} \mathsf{Claim}_{main} &= \mathsf{WellDef}(\Omega_N) \wedge \mathsf{CarrierSep} \wedge \mathsf{MorphismPreserve} \\ &\quad \wedge \mathsf{EffectTyped} \wedge \mathsf{EvidenceScoped}_\Gamma, \\ \mathsf{CryptoCarrier} \cap R &= \varnothing. \end{aligned}$$

Claims outside this boundary are stratified as

$$\mathsf{Claim} = \mathsf{Spec} \cup \mathsf{Impl} \cup \mathsf{Evidence}_\Gamma \cup \mathsf{Obligation}.$$

I. Introduction

Centralized platforms are efficient deployment mechanisms but poor authority carriers: identity, naming, storage, and traffic control collapse to service operators [1]–[3]. Rings asks a narrower question than how to design another overlay:

$$\begin{aligned} \mathsf{Own}(d) &= k_d, \\ \mathsf{Place}_\nu(x) &= \mathrm{owner}_N(H_\nu(x)), \\ \mathsf{Run}(\mathcal{P}_\nu) &= \tau_\nu : S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu), \\ \mathsf{RootOfAuthority} &\neq \mathsf{ApplicationServer}. \end{aligned}$$

Chord and CorrectChord supply the routing substrate; DID-style proofs supply self-certifying identity; WebRTC and WebAssembly supply runtime interpretation; CRDTs supply one merge algebra. None alone gives a typed law for owner, carrier, and evidence scope:

$$\begin{aligned} \mathsf{Chord/CorrectChord} &\Rightarrow \mathsf{ringLookup} \not\Rightarrow \mathsf{resourceAuthority}, \\ \mathsf{DID} &\Rightarrow \mathsf{keyBoundName} \not\Rightarrow \mathsf{overlayOwner}, \\ \mathsf{WebRTC} &\Rightarrow \mathsf{browserTransport} \not\Rightarrow \mathsf{protocolLaw}, \\ \mathsf{CRDT} &\Rightarrow \mathsf{mergeLaw} \not\Rightarrow \mathsf{placementLaw}. \end{aligned}$$

Ryan J. Kung can be reached at ryankung@ieee.org. Manuscript created February 8, 2023; last updated July 7, 2026.

The contribution is therefore an algebraic composition boundary. The Chord identifier space is the additive cyclic group for placement and interval order; signatures, threshold schemes, encryption, and zero-knowledge protocols remain in their own fields or groups:

$$\begin{aligned} R &= \mathbb{Z}/2^{160}\mathbb{Z}, \\ \mathsf{CC} &\in \{\mathbb{F}_q, \mathbb{G}, \mathbb{Z}_q\}, \\ R \cap \mathsf{CC} &= \varnothing. \end{aligned}$$

This paper studies the algebra that makes the implementation coherent, then marks which claims are implemented, model checked in a finite scope, or still benchmark obligations.

II. Problem Statement and Design Goals

Existing decentralized systems solve orthogonal subproblems. The missing object is the commuting authority layer between identity, placement, execution, and evidence:

$$\begin{aligned} \mathcal{N}_R &= \{(X_\nu, H_\nu)\}_\nu, \qquad \mathcal{P}_R = \{\mathcal{P}_\nu\}_\nu, \\ \mathsf{Input} &= (\Pi, \mathcal{R}_N, \mathcal{N}_R, \mathcal{P}_R, \mathcal{X}), \\ \mathsf{Need} &= \mathsf{Owner} : X_\nu \to N \quad \text{and} \quad \mathsf{Run} : \mathcal{P}_\nu \to \mathsf{Effects}, \\ \mathsf{Forbidden} &= \mathsf{CryptoCarrier} = R \ \vee \ \mathsf{Evidence}_\Gamma = \mathsf{UniversalClaim}. \end{aligned}$$

Obligation 1 (Design target).

$$\begin{aligned} G_{id} &= \mathsf{SelfCertID} \wedge \mathsf{DelegatedSession}, \\ G_{own} &= \mathsf{StructuredOwner}, \\ G_{io} &= \mathsf{BrowserNativeTransport}, \\ G_{eff} &= \mathsf{TypedProtocolEffect}, \\ G_{cr} &= \mathsf{CarrierCorrectCrypto}, \\ G_{rep} &= \mathsf{BoundedRepairEvidence}, \\ G_{bd} &= \mathsf{ExplicitClaimBoundary}, \\ \mathsf{Rings} &\models G_{id} \wedge G_{own} \wedge G_{io} \wedge G_{eff} \\ &\quad \wedge G_{cr} \wedge G_{rep} \wedge G_{bd}. \end{aligned}$$

Definition 1 (Methodology and claim ledger).

$$\begin{aligned} \mathsf{Method} &= \mathsf{Mot} \to \mathsf{Spec} \to \mathsf{Algebra} \to \mathsf{ImplMap} \\ &\quad \to \mathsf{Evidence}_\Gamma \to \mathsf{Obligation}, \\ \Gamma &= (|N|, \mathsf{topology}, \mathsf{scheduler}, \mathsf{fault}), \\ \mathsf{ClaimKind} &= \mathsf{Spec} \cup \mathsf{Impl} \cup \mathsf{Evidence}_\Gamma \cup \mathsf{Obligation}, \\ \mathsf{Admit}(c) &\Rightarrow c \in \mathsf{ClaimKind}. \end{aligned}$$

Definition 2 (Global symbols and non-claims).

$$\begin{aligned} \mathsf{NS} &= \{D, V, M, H, U, Q, W\}, \qquad VID = V, \\ \mathsf{Time} &= \mathbb{T}, \quad \mathsf{Ctx} = B^*, \quad \mathsf{Cap} \subseteq A^*, \\ \mathsf{Domain} &= \mathsf{WitnessDomain}, \quad \mathsf{Order} = \{<, =, >, \bot\}, \\ \mathsf{Prov} &= (\mathsf{src}, \mathsf{rev}, \mathsf{cmd}, \mathsf{seed}, \mathsf{host}), \\ \mathsf{Limit} &= \mathsf{NonClaim}_{Rings} \cup \mathsf{Obligation}. \end{aligned}$$

$$\begin{aligned} \mathsf{NonClaim}_{Rings} = \mathsf{new}\{&\mathsf{DHT}, \mathsf{DID}, \mathsf{WebRTC}, \mathsf{CRDT}, \mathsf{Crypto}\} \\ &\cup \{\mathsf{ByzLive}, \mathsf{Consensus}, \mathsf{WANPerf}\}. \end{aligned}$$

Definition 3 (Contribution).

$$\begin{aligned} \mathsf{Contribution}_{Rings} = \{&(R, \mathbf{Set}/R, \Omega_N),\ \mathcal{R}_N,\ \mathsf{RepKey}^\rho, \\ &\mathsf{T}_A^{\mathcal{E}},\ \mathsf{CarrierSep},\ \mathsf{Explore}_\Gamma,\ \mathsf{Report}_{Rings}\}. \end{aligned}$$

$$\mathsf{Reuse}_{Rings} = \{\mathsf{Chord}, \mathsf{CorrectChord}, \mathsf{DID}, \mathsf{WebRTC}, \mathsf{CRDT}\}.$$

Proposition 1 (Main property boundary). Assume $N \neq \varnothing$, the CorrectChord stable-base obligations for $\mathcal{R}_N$, carrier separation, and typed protocol morphisms. Write $\Sigma_{\nu\mu} = (\sigma_X, \sigma_S, \sigma_I, \sigma_O, \sigma_\mathcal{E}, \sigma_A^*)$ and $\mathsf{PMor}_{\nu\mu}(\Sigma_{\nu\mu})$ for the laws of Definition 17. The main claim is:

$$\begin{aligned} \mathsf{OwnerLaw} &\equiv \forall \nu \in \mathsf{NS}. \forall x \in X_\nu.\ P_\nu^N(x) = \mathrm{owner}_N(H_\nu(x)) \in N, \\ \mathsf{MorphLaw} &\equiv \forall \nu, \mu \in \mathsf{NS}. \forall \Sigma_{\nu\mu}. \\ &\qquad \mathsf{PMor}_{\nu\mu}(\Sigma_{\nu\mu}) \Rightarrow P_\mu^N \circ \sigma_X = P_\nu^N, \\ \mathsf{EffectLaw} &\equiv \forall \nu \in \mathsf{NS}. \forall s, i.\ \tau_\nu(s, i) \in Y_\nu, \\ &\quad Y_\nu = \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu), \\ \mathsf{LedgerLaw} &\equiv \forall c.\ c = \mathsf{Evidence}_\Gamma \Rightarrow \mathsf{finite}(\Gamma) \wedge \mathsf{declared}(\Gamma), \\ \mathsf{Main}_{Rings} &\equiv \mathsf{OwnerLaw} \wedge \mathsf{MorphLaw} \wedge \mathsf{EffectLaw} \wedge \mathsf{LedgerLaw}. \end{aligned}$$

It does not entail any $c \in \mathsf{NonClaim}_{Rings}$:

$$\forall c \in \mathsf{NonClaim}_{Rings}. \quad \mathsf{Main}_{Rings} \nvdash c.$$

III. Algebraic System Model

Assumption 1 (System and adversary scope).

$$\begin{aligned} \mathsf{fault}_{overlay} &= \mathsf{fail\text{-}stop}, \\ \mathsf{Adv}_{net} &= \{\mathsf{delay}, \mathsf{drop}, \mathsf{dup}, \mathsf{reorder}, \mathsf{replay}\}, \\ \mathsf{Adv}_{id} &= \{\mathsf{manyDID}, \mathsf{stolenSession}, \mathsf{staleProof}\}, \\ \mathsf{Forge}_\Pi &= \bot, \qquad \mathsf{Preimage}_H = \bot, \\ \mathsf{consensus} &\notin \mathsf{Overlay}, \\ \mathsf{order} &= \mathsf{SidecarWitness}, \\ \mathsf{crypto\_ops} &\cap \mathsf{route\_ops} = \varnothing. \end{aligned}$$

Thus CorrectChord is the safety root only for crash-scoped overlay maintenance. Byzantine routing, Eclipse resistance, Sybil resistance, anonymity, and denial-of-service resistance are not inherited from Chord; they must be discharged by admission, session typing, ranking, relay, quota, and measurement predicates at their own layers.

Definition 4 (Identifier carrier).

$$\begin{aligned} M &= 2^{160}, \\ R &= \mathbb{Z}/M\mathbb{Z}, \\ \delta_a(x) &= (x - a) \bmod M, \\ (a, b]_R &= \{x \in R : 0 < \delta_a(x) \leq \delta_a(b)\}, \\ \mathsf{RouteOps}_R &= \{0, +, -, \delta_a, (\_, \_]_R\}, \\ \mathsf{Mult}_R &\notin \mathsf{RouteOps}_R. \end{aligned}$$

Definition 5 (Live topology).

$$\begin{aligned} N &\subset R, \quad 1 \leq |N| < \infty, \\ \mathrm{owner}_N(k) &= \arg\min_{u \in N} \delta_k(u), \\ \mathrm{cw}_N(n) &= \mathrm{sort}_{\delta_n}(N \setminus \{n\}) \cdot \langle n \rangle, \\ \mathrm{succ}_N(n) &= \mathrm{head}(\mathrm{cw}_N(n)), \\ \mathrm{pred}_N(n) &= \begin{cases} \bot, & |N| = 1, \\ \arg\min_{p \in N \setminus \{n\}} \delta_p(n), & |N| > 1, \end{cases} \\ \mathrm{Succ}_N^q(n) &= \mathrm{take}_{\min(q, |N|)}(\mathrm{cw}_N(n)). \end{aligned}$$

The list $\mathrm{cw}_N(n)$ contains strictly clockwise live nodes and uses n only as the wrap-around sentinel. Hence a singleton ring has $\mathrm{Succ}_N^q(n) = \langle n \rangle$ and $\mathrm{pred}_N(n) = \bot$.

Assumption 2 (Correct-Chord stable base).

$$\begin{aligned} r &\in \mathbb{N}_{>0}, \\ \mathsf{Steady}(N, r) &\Rightarrow |N| \geq r + 1, \\ \mathsf{Init}(N, r) &\Leftrightarrow 1 \leq |N| \leq r, \\ |\mathrm{Succ}_N^r(n)| &= \min(r, |N|), \\ \mathsf{Maintain} &= \{\mathsf{join}, \mathsf{rectify}\} \\ &\quad \cup \{\mathsf{stabilize}\}_{CorrectChord}. \end{aligned}$$

CorrectChord claims in this paper are stable-ring claims under $\mathsf{Steady}(N, r)$; singleton and small-ring behavior is an explicit initialization edge case, not evidence for r distinct live successors.

Definition 6 (Affine replica set).

$$\begin{aligned} 1 \leq \rho &\leq M, \\ \mathrm{repkey}_i^\rho(k) &= (k + \lfloor M \cdot i/\rho \rfloor) \bmod M, \quad 0 \leq i < \rho, \\ \mathrm{RepKey}^\rho(k) &= \langle \mathrm{repkey}_i^\rho(k) : 0 \leq i < \rho \rangle, \\ \mathrm{RepOwner}_N^\rho(k) &= \langle \mathrm{owner}_N(\mathrm{repkey}_i^\rho(k)) : 0 \leq i < \rho \rangle. \end{aligned}$$

$$\begin{aligned} P_k &= \mathrm{set}(\mathrm{RepKey}^{\rho}(k)), \\ O_k &= \mathrm{set}(\mathrm{RepOwner}^{\rho}_N(k)), \\ |P_k| &= \rho, \\ 1 \le |O_k| &\le \min(\rho, |N|), \\ \mathsf{collapse}_{N,\rho}(k) &\Leftrightarrow |O_k| < \min(\rho, |N|). \end{aligned}$$

Replica keys specify intended placement cardinality; live owner images specify realizable redundancy under the current live topology. CorrectChord successor lists remain the topology safety root.

Definition 7 (Rings overlay object).

$$\mathcal{R}_N = (R, N, \mathrm{owner}_N, \mathrm{pred}_N, \mathrm{Succ}^r_N, \mathrm{RepOwner}^{\rho}_N, F, E),$$

$$\begin{aligned} F &\subseteq N \times \mathbb{N} \times N, \\ E &= \textstyle\prod_{\nu \in \mathsf{NS}} E_\nu, \\ \mathsf{SafetyRoot}(\mathcal{R}_N) &= (\mathrm{pred}_N, \mathrm{Succ}^r_N), \\ \mathsf{PerfWitness}(\mathcal{R}_N) &= F. \end{aligned}$$

Proposition 2 (Rotation equivariance).

$$t \in R, \quad N + t = \{n + t : n \in N\} \Rightarrow$$
$$\mathrm{owner}_{N+t}(k + t) = \mathrm{owner}_N(k) + t.$$

Proof.

$$\begin{aligned} \forall n \in N. \quad \delta_{k+t}(n + t) &= ((n + t) - (k + t)) \bmod M \\ &= \delta_k(n). \end{aligned}$$

□

Constraint 1 (Carrier separation).

$$\begin{aligned} \mathsf{PlacementOps} &= \{0, +, -, \delta_a, (a, b]_R\} \\ &\quad \cup \{\mathrm{owner}_N\} \\ &\quad \cup \{\mathrm{RepKey}^{\rho}, \mathrm{RepOwner}^{\rho}_N\}, \\ \mathsf{CryptoOps} &= \{\times, ^{-1}, \cdot_{scalar}, \mathsf{interp}\} \\ &\quad \cup \{\mathsf{pair}, \mathsf{subgroup}\}, \\ \mathsf{PlacementOps} &\cap \mathsf{CryptoOps} = \varnothing, \\ \mathsf{CryptoCarrier} &\in \{\mathbb{F}_q, \mathbb{G}, \mathbb{Z}_q\}. \end{aligned}$$

Object	Carrier or law
Chord IDs	additive cyclic group R
DID proof	signature verifier and transcript law
VID placement	$H_\nu : X_\nu \to R$ then owner map
Storage merge	join semilattice or explicit finalizer
E2E encryption	group operation plus byte-to-point adapter
Sequencing	partial order over witness domains
Effects	writer monad over action lists

Table I
Carrier boundaries in the Rings model

IV. Architecture As Interfaces

Definition 8 (Layer tuple).

$$\mathcal{A} = (I, T, O, X, P, L)$$

I	$(DID, \Pi, \mathrm{Session})$
T	$(C, A_T, \mathsf{offer}, \mathsf{answer}, \mathsf{send}, \mathsf{recv})$
O	$\mathcal{R}_N$
X	$(S_X, A_X, \mathsf{cap}, I_X)$
P	$\{\mathcal{P}_\nu\}_{\nu \in \mathsf{NS}}$
L	$\mathsf{App} \circ P$

The interface tuple $\mathcal{A}$ separates identity, transport, overlay, extension runtime, protocol, and application layers. Only the overlay owns $\mathcal{R}_N$; the other layers discharge typed obligations through I, T, X, P.

Definition 9 (Identity proof).

$$\Pi = (K, \Sigma, V, \mathrm{did}, \mathsf{enc}, \mathsf{ttl})$$

$$\begin{aligned} V &: K \times \Sigma \times B^* \to \{\top, \bot\}, \\ \mathrm{did} &: K \to X_D, \\ \mathsf{enc} &: \Sigma \to B^*, \\ \mathsf{ttl} &: \Sigma \to \mathsf{Time}. \end{aligned}$$

Invariant 1 (Typed session authority).

$$sid = (d, s, e, ctx, cap), \qquad d \in X_D, \quad s \in K, \quad now < e.$$

$$\begin{aligned} \mathrm{Session}(d, s, e, ctx, cap) \Leftrightarrow V(k_d, \sigma_{d \to s}, d \| s \| e \| ctx \| cap) = \top \\ \wedge\ \mathrm{KeyOf}(d) = k_d \wedge \mathrm{did}(k_d) = d. \end{aligned}$$

$$\begin{aligned} \mathsf{HopCheck}(sid) &\equiv \mathrm{Session}(d, s, e, ctx, cap), \\ \mathsf{BoundCtx}(sid) &= (sid, \nu, path, dst), \\ \mathsf{FrameCtx}(f) &= (sid, \nu, path, dst), \\ \mathrm{Accept}_\Delta(f) &\Rightarrow \mathrm{SessionValid}(sid) \\ &\quad \wedge \mathsf{FrameCtx}(f) = \mathsf{BoundCtx}(sid). \end{aligned}$$

Definition 10 (Transport interface).

$$T = (C, \mathsf{offer}, \mathsf{answer}, \mathsf{send}, \mathsf{recv})$$

$$A_T = \{\mathsf{Offer}, \mathsf{Answer}, \mathsf{Ice}, \mathsf{Open}, \mathsf{Send}, \mathsf{Recv}, \mathsf{Close}\}.$$

$$I_T : A_T^* \to \mathsf{IO}_{WebRTC} \cup \mathsf{IO}_{Native}.$$

WebRTC signaling is a runtime adapter rather than a new overlay primitive. Rings may batch offer, answer, and ICE material into fewer protocol actions, but correctness remains delegated to the WebRTC/ICE state machines; evaluation therefore reports connection setup separately from overlay lookup.

Definition 11 (Runtime interpreter).

$$\mathcal{X} = (S_X, A_X, \mathsf{cap}, I_X)$$

$$\begin{aligned} \mathsf{cap} &\subseteq A_X, \\ I_X &: A_X^* \to \mathsf{IO}, \\ I_X(\epsilon) &= \mathsf{Noop}, \\ I_X(\alpha \cdot \beta) &= I_X(\alpha); I_X(\beta). \end{aligned}$$

WebAssembly is modeled only through I_X and cap: it interprets effect lists under runtime capabilities and does not change the overlay object.

Obligation 2 (Runtime confinement).

$$\begin{aligned} \tau &: S \times I \to \mathcal{E} + (S \times O \times A_X^*), \\ \mathsf{emit}(\tau(s, i)) &\subseteq \mathsf{cap}, \\ a \notin \mathsf{cap} &\Rightarrow I_X(a) = \mathsf{Reject}(a). \end{aligned}$$

V. Category and Effects

Definition 12 (Namespace object).

$$\begin{aligned} \nu &\in \mathsf{NS}, \\ H_\nu &: X_\nu \to R, \\ (X_\nu, H_\nu) &\in \mathbf{Set}/R. \end{aligned}$$

Definition 13 (Placement functor).

$$\begin{aligned} \Omega_N &: \mathbf{Set}/R \to \mathbf{Set}/N, \\ (X, H) &\mapsto (X, \mathrm{owner}_N \circ H). \end{aligned}$$

$$P_\nu^N = \mathrm{owner}_N \circ H_\nu : X_\nu \to N.$$

Proposition 3 (Placement functoriality).

$$f : X_\nu \to X_\mu, \qquad H_\mu \circ f = H_\nu.$$

$$P_\mu^N \circ f = P_\nu^N.$$

Proof.

$$\mathrm{owner}_N \circ H_\mu \circ f = \mathrm{owner}_N \circ H_\nu.$$

□

Definition 14 (Writer effect monad).

$$\begin{aligned} A^* &= \mathsf{FreeMonoid}(A), \\ \mathsf{T}_A(X) &= X \times A^*, \\ \eta(x) &= (x, \epsilon), \\ (x, \alpha) \gg\!= f &= \mathsf{let}\ f(x) = (y, \beta)\ \mathsf{in}\ (y, \alpha \cdot \beta). \end{aligned}$$

Proposition 4 (Effect associativity).

$$(h^\star \circ g^\star) \circ f^\star = h^\star \circ (g^\star \circ f^\star).$$

Proof.

$$((\alpha \cdot \beta) \cdot \gamma) = (\alpha \cdot (\beta \cdot \gamma)).$$

□

Definition 15 (Typed failure stack).

$$\mathsf{T}_A^{\mathcal{E}}(X) = \mathcal{E} + (X \times A^*).$$

$$\begin{aligned} \pi_A(e \in \mathcal{E}) &= \epsilon, \\ \pi_A(x, \alpha) &= \alpha, \\ \mathsf{RecoverableReport} &\in A. \end{aligned}$$

Definition 16 (Protocol object).

$$\mathcal{P}_\nu = (X_\nu, S_\nu, I_\nu, O_\nu, A_\nu, \mathcal{E}_\nu, H_\nu, \tau_\nu)$$

$$\tau_\nu : S_\nu \times I_\nu \to \mathsf{T}_{A_\nu}^{\mathcal{E}_\nu}(S_\nu \times O_\nu).$$

Definition 17 (Protocol morphism).

$$\Sigma : \mathcal{P}_\nu \to \mathcal{P}_\mu = (\sigma_X, \sigma_S, \sigma_I, \sigma_O, \sigma_\mathcal{E}, \sigma_A^*)$$

$$\begin{aligned} \sigma_A^* &: A_\nu^* \to A_\mu^*, \\ \sigma_A^*(\alpha \cdot \beta) &= \sigma_A^*(\alpha) \cdot \sigma_A^*(\beta), \\ H_\mu \circ \sigma_X &= H_\nu, \\ \mathsf{T}_{\sigma_A}^{\sigma_\mathcal{E}}(\sigma_S \times \sigma_O) \circ \tau_\nu &= \tau_\mu \circ (\sigma_S \times \sigma_I). \end{aligned}$$

VI. Chord Overlay

$$\begin{aligned} \mathsf{Base} &= \mathsf{Chord}, \\ \mathsf{Maintenance} &= \mathsf{CorrectChord}, \\ \mathsf{Object} &= \mathcal{R}_N, \\ \mathsf{Witnesses} &= \{\text{Algorithms } 1, 2, 3\}. \end{aligned}$$

This section fixes Chord as the base protocol and CorrectChord as the maintenance law [4], [5].

Definition 18 (Local node state and Chord auxiliaries).

$$\begin{aligned} S_n &= (p_n, L_n, F_n, E_n), \\ p_n &\in N \cup \{\bot\}, \quad L_n \in N^{\le r}, \\ F_n &: \{0, \dots, 159\} \rightharpoonup N. \end{aligned}$$

$$\begin{aligned} \mathsf{head}(\langle\rangle) &= \bot, \\ \mathsf{Normalize}_r(n, X) &= \mathrm{take}_{\min(r, |N|)}(U_n(X)), \\ U_n(X) &= \mathrm{uniq}(\mathrm{filter}_N(X) \cdot \mathrm{cw}_N(n)), \\ \mathrm{target}_i(n) &= n + 2^i \bmod M, \\ \mathrm{finger}_i(n) &= \mathrm{owner}_N(\mathrm{target}_i(n)), \\ \mathsf{Cand}_S(n, k) &= (\mathrm{rng}(F_n) \cup \mathrm{set}(L_n)) \cap (n, k)_R, \\ \mathsf{ClosestPreceding}_S(n, k) &= \max_{\delta_n} \mathsf{Cand}_S(n, k). \end{aligned}$$

Here $\mathsf{ClosestPreceding}_S(n, k) = \bot$ when the displayed set is empty.

Algorithm 1 DHT Join and Rectify Transitions

```
function BeginJoin(n, known)
    pred[n] ← ⊥
    if known = ⊥ then
        succ[n] ← ⟨n.did⟩
        return Done
    return Remote(known, FindSuccessor(n.did))
function InstallSuccessor(n, s)
    if s = ⊥ then
        return Repair(DiscoverSuccessor(n.did))
    succ[n] ← Normalize_r(n, ⟨s⟩)
    return Remote(s, QuerySuccList)
function InstallSuccList(n, s, list)
    succ[n] ← Normalize_r(n, ⟨s⟩ · list)
    h ← head(succ[n])
    if h = ⊥ then
        return Repair(DiscoverSuccessor(n.did))
    else if h = n.did then
        return Done
    else
        a ← Remote(h, Notify(n.did))
        b ← Remote(h, SyncStorage(n.did))
        return Multi(a, b)
function HandleNotify(n, q)
    if q ≠ n.did ∧ (pred[n] = ⊥ ∨ q ∈ (pred[n], n.did)_R) then
        pred[n] ← q
    return Noop
```

Algorithm 2 DHT Lookup Algorithm

```
1: function FindSuccessor(key, n)
2:     if key = n.did then
3:         return Found(n.did)
4:     s ← head(succ[n])
5:     if s = ⊥ then
6:         return Repair(QuerySuccList(n.did))
7:     else if s = n.did then
8:         return Found(n.did)
9:     else if key ∈ (n.did, s]_R then
10:        return Found(s)
11:    next ← ClosestPreceding_S(n.did, key)
12:    if next = ⊥ or next = n.did then
13:        next ← s
14:    return Remote(next, FindSuccessor(key))
```

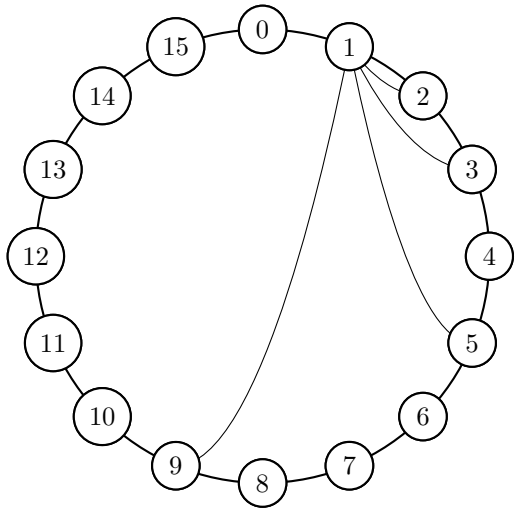

Figure 1. Chord finger targets from node 1 in a 16-node ring

Invariant 2 (Ring fixpoint).

$$\begin{aligned}\mathrm{Inv}_{ring}(N,S) \equiv \forall n \in N.\ & p_n = \mathrm{pred}_N(n) \\ & \wedge L_n = \mathrm{Succ}^r_N(n) \\ & \wedge \mathsf{head}(L_n) = \mathrm{succ}_N(n).\end{aligned}$$

Invariant 3 (Lookup owner preservation).

$$\mathsf{Eval}_S(n,k) \Downarrow o$$

be the fair closure of Algorithm 2, where $Remote(m, FindSuccessor(k))$ resumes the same lookup at node m. Then

$$\begin{aligned}&(\mathsf{Steady}(N,r) \vee |N| = 1) \wedge \mathrm{Inv}_{ring}(N,S) \\ &\Rightarrow \forall n \in N. \forall k \in R.\ \mathsf{Eval}_S(n,k) \Downarrow \mathrm{owner}_N(k).\end{aligned}$$

Proof obligation. If Algorithm 2 returns $Found(s)$ by the interval test, then $s = head(L_n) = \mathrm{succ}_N(n)$ and no live node lies in $(n,s)_R$; therefore $\mathrm{owner}_N(k) = s$. If it forwards, the chosen target is either the closest preceding finger/list entry or the first successor; in both cases it lies in $(n,k)_R$, so $\delta_{next}(k) < \delta_n(k)$. Finiteness of N gives termination. The branches $key = n.did$ and $s = n.did$ discharge the singleton and exact-owner edge cases. Repair branches are outside the steady-ring premise. □

Algorithm 3 DHT Stabilization

```
1: function BeginStabilize(n)
2:     s ← head(succ[n])
3:     if s = ⊥ then
4:         return Repair(DiscoverSuccessor(n.did))
5:     else if s = n.did then
6:         return Noop
7:     else
8:         return Remote(s, QueryTopology)
9: procedure Stabilize(n, topo)
10:    actions ← []
11:    s ← head(succ[n])
12:    if s = ⊥ then
13:        return Repair(DiscoverSuccessor(n.did))
14:    else if s = n.did then
15:        return Noop
16:    c ← topo.pred
17:    if c ≠ ⊥ and c ≠ n.did and c ∈ (n.did, s)_R then
18:        succ[n] ← Normalize_r(n, ⟨c⟩ · succ[n])
19:    succ[n] ← Normalize_r(n, succ[n] · topo.succ)
20:    h ← head(succ[n])
21:    if h ≠ ⊥ and h ≠ n.did then
22:        a ← Remote(h, Notify(n.did))
23:        b ← Remote(h, QuerySuccList)
24:        actions ← actions · [a, b]
25:    return Multi(actions)
```

VII. DID and Resource Algebra

Definition 19 (DID namespace).

$$\mathcal{D} = (X_D, H_D, \Pi, \mathrm{Session}), \qquad H_D : X_D \to R.$$

$$\begin{aligned}D_K(k) &= H_D(\mathrm{did}(k)), \\ \mathrm{owner}_N(D_K(k)) &\in N, \\ \mathrm{Session} &\subseteq X_D \times K \times \mathsf{Time} \times \mathsf{Ctx} \times \mathsf{Cap}.\end{aligned}$$

Definition 20 (VID namespace).

$$\begin{aligned}\mathcal{V} &= (X_V, H_V, \mathrm{Auth}_V), \\ H_V &: X_V \to R, \\ \mathrm{PrivateKey}(H_V(x)) &= \bot, \\ \mathrm{Auth}_V(op, x, \sigma, caps) &\in \{\top, \bot\}.\end{aligned}$$

Definition 21 (Biased order).

$$A \preceq_C B \iff \delta_C(A) \le \delta_C(B).$$

Definition 22 (Replicated entry algebra).

$$\begin{aligned}\mathcal{E}^{join}_v &= (E_v, \sqsubseteq, \sqcup, \bot), \\ \mathcal{E}^{final}_v &= (E_v, \mathsf{conflict}_v, \mathsf{finalize}_v).\end{aligned}$$

$$\begin{aligned}x \sqcup y &= y \sqcup x, \\ (x \sqcup y) \sqcup z &= x \sqcup (y \sqcup z), \\ x \sqcup x &= x, \\ x \sqcup \bot &= x, \\ \mathsf{finalize}_v &: \mathsf{conflict}_v(E_v) \to E_v.\end{aligned}$$

Algorithm 4 Biased Circular Comparison

```
function BiasedCompare(A, B, C)
    a ← δ_C(A)
    b ← δ_C(B)
    if a < b then
        return A <_C B
    else if a > b then
        return B <_C A
    else
        return A = B
```

$$\mathsf{case}_1 = \mathsf{CRDT}.$$

This is the CRDT join-semilattice case [6].

Invariant 4 (Placement).

$$\begin{aligned} P_{\nu,x} &= \mathrm{set}(\mathrm{RepKey}^{\rho}(H_{\nu}(x))),\\ K^{impl}_{\nu,x} &\subseteq R,\\ \mathrm{Inv}_{place} &\equiv \forall \nu, x.\ K^{impl}_{\nu,x} \subseteq P_{\nu,x}. \end{aligned}$$

Obligation 3 (Replica realization).

$$\begin{aligned} O_{\nu,x} &= \mathrm{set}(\mathrm{RepOwner}^{\rho}_{N}(H_{\nu}(x))),\\ L_{\nu,x} &= |O_{\nu,x}|,\\ \mathsf{FullRed}_{\nu,x} &\Leftrightarrow L_{\nu,x} = \min(\rho, |N|),\\ \mathsf{Collapse}_{\nu,x} &\Leftrightarrow L_{\nu,x} < \min(\rho, |N|),\\ \mathsf{LiveClaim}_{\nu,x}(\rho) &\Rightarrow \mathrm{Inv}_{place} \wedge \mathsf{FullRed}_{\nu,x},\\ \mathsf{Report}_{Rings} &\ni |\{(\nu, x) : \mathsf{Collapse}_{\nu,x}\}|. \end{aligned}$$

When $|N| < \rho$, fewer than ρ distinct live owners is not collapse; collapse means fewer owners than the live topology can realize.

Definition 23 (Mailbox).

$$\begin{aligned} mbox(d, e) &= H_M(\texttt{rings.mailbox} \| d \| e),\\ entry_M &= (\eta, exp, proof_s, dst, cipher). \end{aligned}$$

Definition 24 (Hidden service descriptor).

$$\begin{aligned} h &= (\nu, \rho, Intro, Relay,\\ &\quad \mathsf{Cap}, K, exp, rank),\\ \mathrm{vid}(h) &= H_H(h),\\ \mathsf{Lookup}(h) &= P^N_H(h),\\ \mathsf{Valid}(h, \sigma) &= \mathrm{Auth}_H(h, \sigma). \end{aligned}$$

Definition 25 (Subring).

$$\mathcal{S}_u = (u, N_u, \mathrm{owner}_{N_u}, \mathrm{Succ}^{r_u}_{N_u}, E_u)$$

$$\begin{aligned} u &= H_U(policy),\\ memberlog_u &\in \mathsf{MergeLog} \cup \mathsf{FinalState}. \end{aligned}$$

VIII. Cryptographic Carriers and Traffic

Definition 26 (End-to-end carrier).

$$\begin{aligned} |\mathbb{G}| &= q,\\ \mathsf{Scalar}(\mathbb{G}) &= \mathbb{Z}_q,\\ x &\in \mathbb{Z}_q,\\ h &= g^x,\\ \mathbb{G} &\neq R. \end{aligned}$$

Algorithm 5 is the group-level carrier operation. Byte payloads must first be encoded into curve points and split into bounded frames. The direct ElGamal body is malleable when detached from its signed transport envelope; Rings therefore treats message signatures, DID ownership of the public key, stream identifiers, sequence numbers, and final-frame checks as part of the security boundary rather than as optional metadata.

Algorithm 5 ElGamal Encryption and Decryption

```
Input: Encoded group element m_G ∈ 𝔾, private key
       x ∈ ℤ_q, public key h = g^x, generator g
Output: Group ciphertext (c_1, c_2)
procedure Encryption
    Choose a random integer k ∈ ℤ_q
    Compute c_1 = g^k
    Compute c_2 = m_G · h^k
    return (c_1, c_2)
procedure Decryption
    Compute m_G = c_2 · (c_1)^{-x}
    return m_G
```

Definition 27 (Signed encrypted frame).

$$\begin{aligned} f &= (sid, seq, last, cipher, pk_s, \sigma_s).\\ &\quad sid = (d, s, e, ctx, cap), \qquad d \in X_D, \quad s \in K,\\ &\quad m_f = sid \| seq \| last \| cipher,\\ \mathrm{FrameValid}(f) &\Leftrightarrow V(pk_s, \sigma_s, m_f) = \top\\ &\qquad \wedge\ \mathrm{Session}(d, s, e, ctx, cap)\\ &\qquad \wedge\ pk_s = s\\ &\qquad \wedge\ \mathsf{FrameCtx}(f) = \mathsf{BoundCtx}(sid). \end{aligned}$$

Definition 28 (Receiver sequence policy).

$$\Delta = (next, final, closed, pending, seen, w).$$

$$\begin{aligned} \mathsf{NoFuture}_{\Delta}(seq) &\Leftrightarrow \neg\exists p \in pending.\ p > seq,\\ \mathsf{Dup}_{\Delta}(seq) &\Leftrightarrow seq \in seen \cup pending,\\ \mathsf{New}_{\Delta}(seq, last) &\Leftrightarrow seq \notin seen \cup pending\\ &\quad \wedge \neg closed\\ &\quad \wedge seq - next \leq w\\ &\quad \wedge (final = \bot \vee seq \leq final)\\ &\quad \wedge (last \Rightarrow \mathsf{NoFuture}_{\Delta}(seq)),\\ \mathrm{Accept}_{\Delta}(f) &\Leftrightarrow \mathrm{FrameValid}(f) \wedge \mathsf{New}_{\Delta}(seq, last),\\ \mathrm{DropDup}_{\Delta}(f) &\Leftrightarrow \mathrm{FrameValid}(f) \wedge \mathsf{Dup}_{\Delta}(seq) \wedge \mathsf{emit} = \epsilon. \end{aligned}$$

Here w is receiver policy, not a signed wire field. A duplicate frame may be valid, but it is not accepted as new input.

Definition 29 (Secret sharing placement).

$$\begin{aligned} P &\in \mathbb{F}_q[X],\\ sh_i &= (i, P(i)) \in \mathbb{F}_q^2,\\ H_S(sh_i) &\in R,\\ \mathrm{reconstruct}(\{sh_i\}) &= P(0) \quad (\mathbb{F}_q\text{-interp.}) \end{aligned}$$

This is Shamir reconstruction over the separate field carrier [7].

Definition 30 (Relay frame).

$$r = (path, dst, \nu, session, payload, report).$$

$$\begin{aligned} \mathrm{RelayValid}(r) &= \mathrm{SessionValid}(session) \\ &\quad \wedge\ dst \in R, \\ \mathrm{SplitVocabulary} &= \mathsf{MSRP}. \end{aligned}$$

The split vocabulary follows MSRP [8].

Algorithm 6 End-to-End Chunking Algorithm

```
procedure E2E Chunking
    Input: data, chunk size
    Output: chunks
    chunks ← []
    total-chunks ← ceil(len(data)/chunk-size)
    for i in range(total-chunks) do
        chunk ← data[i*chunk-size : (i+1)*chunk-size]
        append chunk to chunks
    return chunks
```

Invariant 5 (Chunk reassembly).

$$\begin{aligned} \mathrm{complete}(m) \Leftrightarrow \forall i < total(m).\ \exists! c_i \\ \wedge \sum_i |c_i| \le B_m \\ \wedge\ now < ttl(m). \end{aligned}$$

IX. Ordering, Ranking, and Security

Definition 31 (Sidecar witness).

$$w = (domain, root, height, time, finality).$$

$$\begin{aligned} c &= H(msg), \\ \mathrm{Witness}(msg, w) &\Leftrightarrow \mathrm{Include}(c, w.root) = \top \\ &\quad \wedge\ \mathrm{Final}(w) = \top. \end{aligned}$$

Definition 32 (Witness order).

$$\begin{aligned} a \prec b \Leftrightarrow domain(w_a) = domain(w_b) \\ \wedge\ time(w_a) < time(w_b). \end{aligned}$$

$$\chi : \mathsf{Domain} \times \mathsf{Domain} \to \mathsf{Order}.$$

Definition 33 (Ranking event).

$$\begin{aligned} q = (subject, rater, behavior, \\ evidence, epoch, weight, \sigma). \end{aligned}$$

$$\begin{aligned} m_q &= subject \| behavior \| evidence \| epoch, \\ \mathrm{RankValid}(q) &\Leftrightarrow V(k_{rater}, \sigma, m_q) = \top. \end{aligned}$$

Definition 34 (Score aggregation).

$$\begin{aligned} score_e(s) = \mathrm{robust}\{weight(q) \cdot value(q) : \\ q.subject = s\}. \end{aligned}$$

$$\mathrm{robust} \in \{\mathsf{median}, \mathsf{trimmedMean}, \mathsf{declared}\}.$$

$$\begin{aligned} \mathsf{Base} &\not\vdash \mathsf{SybilResist} \wedge \mathsf{EclipseResist} \wedge \mathsf{Anon} \wedge \mathsf{DoSResist}, \\ \mathsf{SecClaim} &\Rightarrow \mathsf{SelfCertID} \wedge \mathsf{TypedSession} \wedge \mathsf{ReplayReject} \\ &\quad \wedge\ \mathsf{Adm} \wedge \mathsf{Rank} \wedge \mathsf{Relay} \wedge \mathsf{Quota}. \end{aligned}$$

Attack	Predicate	Boundary claimed here
Sybil	one actor controls many DIDs	not solved by overlay; admission, ranking, quota
Eclipse	biased neighbor or successor view	not claimed under Byzantine routing; requires secure-routing policy
Replay	stale valid signature or frame	typed session, expiry, nonce/sequence set
MITM	wrong key for DID or session	DID proof, signed envelope, context binding
DoS	unbounded storage, relay, or chunks	quotas, TTL, chunk bounds, backpressure
Privacy	route or relay observation	relay policy only; no anonymity theorem

Table II
Threat predicates and claim boundaries

Obligation 4 (Replay and session-confusion rejection).

$$\begin{aligned} \mathrm{Accept}_\Delta(f,t) &\Rightarrow \mathrm{SessionValid}(sid(f)) \\ &\quad \wedge\ t < exp(sid(f)) \\ &\quad \wedge\ \mathrm{ctx}(f) = \mathrm{ctx}(sid(f)) \\ &\quad \wedge\ \mathrm{fid}(f) \notin seen, \\ seen' &= seen \cup \{\mathrm{fid}(f)\}, \\ \mathrm{emit}(f) &\Rightarrow \mathrm{fid}(f) \in seen', \\ \mathrm{DropDup}_\Delta(f) &\Rightarrow \mathsf{emit} = \epsilon. \end{aligned}$$

X. Verification Evidence and Evaluation Scope

$$\mathsf{Ledger} = \mathsf{Spec} \uplus \mathsf{Evidence}_\Gamma \uplus \mathsf{Baseline}_C \uplus \mathsf{Runtime}_R.$$

This section is an evidence ledger, not a performance claim. The repository contains executable finite-scope evidence for overlay, storage, and message-flow obligations. It does not yet contain a wide-area Rings benchmark, committed benchmark-table regeneration data, or committed seeds sufficient to regenerate the tables below.

$$\begin{aligned} \mathsf{AdmittedEval}(d) &\Leftrightarrow \mathsf{Regen}(d), \\ \neg\mathsf{AdmittedEval}(d) &\Rightarrow d \in \mathsf{ReportedOnly}. \end{aligned}$$

Definition 35 (Execution relation and effect abstraction). Let A_A be concrete scheduled actions and $\mathcal{I}_A$ the abstract inputs of actor A.

$$\begin{aligned} I_A &: A_A \rightharpoonup \mathcal{I}_A, \\ I_A^\sharp(\langle a_1, \ldots, a_m \rangle) &= \langle I_A(a_j) : I_A(a_j) \downarrow \rangle_{j=1}^m. \end{aligned}$$

For triples, lift componentwise:

$$I_A^\sharp(s, o, \beta) = (s, o, I_A^\sharp(\beta)).$$

$$\frac{Q = a :: Q_0 \quad I_A(a) = i \quad \tau_A(S, i) = (S', O, \beta)}{(S, Q) \to_A (S', Q_0 \cdot \beta)}$$

$$\begin{aligned} \mathsf{Explore}_\Gamma &= \mathsf{ModelCheck}(\to_A, \Gamma), \\ \Gamma &= (|N|, \mathsf{topology}, \mathsf{scheduler}, \mathsf{fault}). \end{aligned}$$

$$\mathsf{ModelClaim}(\Gamma) \Rightarrow \mathsf{finite}(\Gamma) \wedge \mathsf{declared}(\Gamma).$$

The relation follows TLA+ style and Stateright execution in the declared finite scope [9], [10].

Definition 36 (Effect preorder).

$$(s, o, \alpha) \preceq_A (s', o', \alpha') \iff s = s' \wedge o = o' \wedge \mathsf{obs}_A(\alpha) = \mathsf{obs}_A(\alpha').$$

Here obs_A erases implementation-local stutter effects such as logging, timers already represented by Γ, and transport bookkeeping not visible in the actor interface.

Invariant 6 (Effect refinement).

$$\forall s, i. \quad I_A^\sharp(\mathrm{Impl}_A(s, i)) \preceq_A I_A^\sharp(\tau_A(s, i)).$$

Equivalently, implementation and specification have equal state/output projections and equal observable abstract effect traces:

$$\pi_{state,out}(I_A^\sharp(\mathrm{Impl}_A)) = \pi_{state,out}(I_A^\sharp \circ \tau_A).$$

Obligation 5 (Topology coverage).

$$\mathcal{L} = \{line, wrap, gap, partition, merge, churn\}$$

$$\mathcal{S} = \{loss, dup, reorder, timeout, restart\}.$$

$$\mathsf{CheckCase} = (L, S, P), \qquad L \in \mathcal{L},\ S \in \mathcal{S}.$$

Artifact	Checked scope	Non-claim / limit
Topology tests	join, rectify, stabilize fixpoints on finite DID layouts	not a parametric TLAPS proof
Stateright	predecessor update, bounded discovery abstraction, storage CRDT, handoff	finite scoped models, not a live-network theorem
Trace replay	deterministic delivery schedules over real routing code	not an Internet latency benchmark
E2E tests	signed direct-ElGamal frame order, final marker, and replay behavior	not a symbolic cryptographic proof
CI witness	build, test, lint, doc, release-package paths	no benchmark-table regeneration job

Table III
Verification evidence and explicit limits. Entries are finite-scope implementation evidence; none is a wide-area Rings lookup benchmark.

Definition 37 (Latency decomposition).

$$T_{total} = T_{conn} + h \cdot T_{hop} + T_{crypto} + T_{app} + T_{witness}.$$

$$h = O(\log |N|) \quad (F_n \text{ populated}).$$

Only hT_{hop} is Chord-comparable; the rest is Rings runtime overhead.

Definition 38 (Benchmark data model).

$$\begin{aligned} \eta &= (N, r, m, L, f, \lambda, \chi, M), \\ M &\in \{\mathsf{maintained}, \mathsf{stale}\}, \\ \pi(d) &\in \mathsf{Prov}, \\ \mathsf{Regen}(d) &\Rightarrow \mathsf{src}, \mathsf{rev}, \mathsf{cmd} \neq \bot, \\ &\quad\ \mathsf{seed}, \mathsf{host} \neq \bot. \end{aligned}$$

$$\begin{aligned} F_C(\eta; D) &\subseteq Q_C(\eta; D), \\ \phi_C(\eta; D) &= |F_C(\eta; D)|/|Q_C(\eta; D)|, \\ ok_C(\eta; D) &= 1 - \phi_C(\eta; D), \\ \bar{to}_C(\eta; D) &= |Q_C|^{-1} \textstyle\sum_{q \in Q_C} to_C(q), \\ B_C(\eta; D) &= (\mathbb{E}[h_C^f \mid \eta, D], \mathbb{E}[h_C^r \mid \eta, D], \bar{to}_C, ok_C, \phi_C), \\ h_C^r &= h_C^f + 1. \end{aligned}$$

$$\begin{aligned} D_{reported} &= D_C^{paper} \uplus D_C^{656} \uplus D_R^{docker}, \\ D_{admit} &= \{d \in D_{reported} : \mathsf{AdmittedEval}(d)\}, \\ D_C^{paper} &= \{C_{II}, C_{III}, C_{IV}, C_8, C_9, C_{10}\}, \\ D_C^{656} &= \{\mathsf{maintained}, \mathsf{stale}\}, \\ D_R^{docker} &= \{D_R^{conv}, D_R^{tx}\}. \end{aligned}$$

$$\begin{aligned} D_C^{paper} &: (N, r) = (1000, 20), \\ D_C^{656} &: (N, r, m, L) = (1600, 3, 160, 64), \\ D_C^{paper} \cup D_C^{656} &\subset \mathsf{Baseline}_C, \\ D_R^{docker} &\subset \mathsf{Runtime}_R, \\ \mathsf{Baseline}_C \cap \mathsf{Runtime}_R &= \varnothing. \end{aligned}$$

Here D_C^{paper} is an external Chord-paper coordinate, D_C^{656} is a PR 656 Chord-only coordinate at Rings' current overlay parameters, and D_R^{docker} is reported Rings runtime evidence. Unless $\mathsf{AdmittedEval}(d)$ holds, a tuple is a reported coordinate only; it is not used to prove Main_{Rings}. Metrics are not pooled unless a Rings lookup benchmark with the same workload key η is added.

Table IV records the Chord-paper baseline coordinates ($N = 1000, r = 20$). It calibrates comparison scale only; it is neither Rings runtime evidence nor generated by this repository.

η	N, r	$\mathbb{E}[h_C^f]$	$\mathrm{timeout}_C$	$10^4\,\mathrm{fail}_C$
$C_{II}, f = 0$	$1000, 20$	3.815	0.000	0.0
$C_{II}, f = .10$	$1000, 20$	4.007	0.522	0.0
$C_{II}, f = .20$	$1000, 20$	4.177	1.071	0.0
$C_{II}, f = .30$	$1000, 20$	4.354	1.830	0.0
$C_{II}, f = .40$	$1000, 20$	4.791	3.700	0.0
$C_{II}, f = .50$	$1000, 20$	5.345	6.565	0.0
$C_{III}, \lambda = .05$	$\simeq 1000, 20$	3.905	0.076	7.0
$C_{III}, \lambda = .10$	$\simeq 1000, 20$	3.949	0.168	19.0
$C_{III}, \lambda = .15$	$\simeq 1000, 20$	4.044	0.272	18.0
$C_{III}, \lambda = .20$	$\simeq 1000, 20$	4.094	0.335	35.0
$C_{III}, \lambda = .25$	$\simeq 1000, 20$	4.128	0.405	36.0
$C_{III}, \lambda = .30$	$\simeq 1000, 20$	4.221	0.501	47.0
$C_{III}, \lambda = .35$	$\simeq 1000, 20$	4.308	0.599	60.0
$C_{III}, \lambda = .40$	$\simeq 1000, 20$	4.345	0.656	58.0

Table IV
External Chord-paper baseline coordinates ($N = 1000, r = 20$), transcribed as reported comparison coordinates. They are not Rings runtime evidence and are not regenerated by the current artifact.

Table V fixes a Chord-only baseline at Rings' current overlay parameters ($N = 1600, r = 3, m = 160, L = 64$). Maintained rows are the comparison baseline; stale rows intentionally disable maintenance and measure sensitivity, not Rings behavior.

Thus stale rows explain sensitivity, not the maintained Chord baseline:

$$\begin{aligned} M = \mathsf{stale} &\Rightarrow \neg\mathsf{maintenance}, \\ M = \mathsf{maintained} &\Rightarrow \mathsf{Baseline}_C^{656}. \end{aligned}$$

Obligation 6 (Benchmark report).

$$\mathsf{Report}_{eval} = (\mathsf{Baseline}_C^{paper}, \mathsf{Baseline}_C^{656}, \mathsf{Runtime}_R^{docker}, \mathsf{Prov}, \mathsf{Limit}).$$

η	model	$\bar{h}^f/\bar{h}^r$	ok	$\bar{to}$	$10^4\phi$
stable	maint.	4.469/5.469	1.000	0.000	0.0
$f = .10$	maint.	4.394/5.394	1.000	0.000	0.0
$f = .20$	maint.	4.316/5.316	1.000	0.000	0.0
$\chi = .05$	maint.	4.465/5.465	1.000	0.000	0.0
$\chi = .40$	maint.	4.470/5.470	1.000	0.000	0.0
$f = .10$	stale	4.758/5.758	.990	.734	98.2
$f = .20$	stale	5.186/6.186	.969	1.890	314.7
$\chi = .05$	stale	4.617/5.617	.945	.361	546.2
$\chi = .40$	stale	5.746/6.746	.484	4.604	5161.1

Table V
Chord-only baselines at Rings parameters $(N = 1600, r = 3, m = 160, L = 64)$. Maintained rows are the baseline; stale rows are no-maintenance sensitivity. These are PR 656 reported coordinates, not admitted current-artifact evidence until generator, revision, seed, command, and host are shipped.

$$\begin{aligned} \mathsf{ReportCmp}_{lookup} &= B_C(\eta; D_C^{paper} \uplus D_C^{656}), \\ \mathsf{Measure}_{runtime} &= B_R^{docker}(\eta_R; D_R^{docker}), \\ \mathsf{ReportCmp}_{lookup} &\not\equiv \mathsf{Measure}_{runtime}. \end{aligned}$$

A Rings performance claim must measure Rings under the reported build; a Chord baseline may only be cited as a baseline.

$$\begin{aligned} \vec{T}_{total} &= (T_{conn}, hT_{hop}, T_{crypto}, \\ &\quad T_{app}, T_{witness}), \\ \mathsf{Ccmp}(\vec{T}_{total}) &= hT_{hop}, \\ \mathsf{Rovh} &= (T_{conn}, T_{crypto}, T_{app}, \\ &\quad T_{witness}, \mathsf{runtime}). \end{aligned}$$

Only hT_{hop} is Chord-comparable. Connection establishment, cryptography, application handling, witness generation, and runtime scheduling are Rings overheads and require Rings measurements.

$$\begin{aligned} D_R^{conv} &= \mathsf{raw}(16, 309.532s, 0/16, 3/16, \\ &\quad .194, 1.026, 8058.6), \\ D_R^{tx} &= \{(1, 10.46), (16, 107.35), \\ &\quad (64, 90.36)\} \subset \mathrm{KiB} \times \mathrm{Mbps}. \end{aligned}$$

The Docker/WebRTC cluster tuple is convergence evidence, not a steady-state lookup benchmark. D_R^{tx} is payload throughput; it must not be compared to Chord forwarding hops or timeout rates.
Artifact provenance.

$$\begin{aligned} \mathsf{AdmittedEval}(d) \Leftrightarrow\ & \mathsf{src}(d) \neq \bot \wedge \mathsf{rev}(d) \neq \bot \\ & \wedge \mathsf{cmd}(d) \neq \bot \wedge \mathsf{seed}(d) \neq \bot \\ & \wedge \mathsf{host}(d) \neq \bot. \end{aligned}$$

No CI command currently regenerates Table IV, Table V, D_R^{conv}, or D_R^{tx}.
Limitations. The current artifact supports finite model/test evidence and reported baseline coordinates. It does not yet support claims about wide-area latency, NAT population behavior, adversarial churn beyond the scoped models, symbolic cryptographic security, or steady-state Rings lookup performance.

XI. Related Work

Self-certifying systems bind names to keys, while DID standardization separates identifier syntax, verification methods, and resolution metadata [11], [12]. Rings follows that line by making DID control a cryptographic predicate, but it places resources through $\Omega_N(X, H) = (X, \mathrm{owner}_N \circ H)$ rather than through a global certificate authority or ledger.

Protection systems and authorization logics make authority an explicit predicate; monads and algebraic effects make runtime effects typed [13]–[16]. Rings uses these tools as composition discipline: authority, placement, and effect emission live in separate carriers and meet only through typed morphism laws.

Structured overlays give different placement laws. Chord gives clockwise successor ownership; Kademlia/IPFS/libp2p give XOR or provider-record lookup; Pastry, Tapestry, and CAN explore prefix, proximity, and coordinate-space routing [4], [17]–[22]. Rings' novelty is not another lookup metric; it is the separation between the circular owner law, DID/session authority, CRDT-style storage merge, and browser/native transport interpretation.

Security work on DHTs shows why this separation must be explicit. Secure routing, Sybil, and Eclipse attacks require admission, identifier assignment, neighbor-set constraints, auditing, or redundant routing beyond ordinary DHT maintenance [23]–[26]. Rings therefore states Sybil, Eclipse, anonymity, and DoS resistance as policy obligations, not as consequences of CorrectChord.

Rings storage uses the join-semilattice CRDT case when merge is available and an explicit finalizer otherwise [6]. WebRTC and WebAssembly are runtime carriers, not overlay algorithms; WebRTC measurement work also suggests that connection setup, NAT traversal, and payload throughput must be reported separately from DHT hop count [27]–[29].

XII. Conclusion

Rings should be read as a bounded theorem about an implemented algebraic object, not as an informal bundle of peer-to-peer mechanisms:

$$\mathsf{Rings} = (R, \mathcal{R}_N, \mathbf{Set}/R, \Omega_N, \mathsf{T}_A^{\mathcal{E}}, \Pi, \mathbb{G}, \mathcal{X}).$$

$$\begin{aligned} \mathsf{Established} &= \mathsf{OwnerFunctor} \wedge \mathsf{CarrierSep} \\ &\quad \wedge \mathsf{TypedEffects} \wedge \mathsf{ImplMap}, \\ \mathsf{ScopedEvidence} &= \mathsf{Explore}_\Gamma \wedge \mathsf{traceReplay} \\ &\quad \wedge \mathsf{declared}(\Gamma), \\ \mathsf{Obligation} &= \mathsf{wideAreaBenchmark} \cup \mathsf{adversarialLiveness} \\ &\quad \cup \mathsf{fullReplicaRealization}, \\ \mathsf{NonClaim} &= \mathsf{NonClaim}_{Rings}. \end{aligned}$$

The scientific contribution is the composition law and its claim ledger: CorrectChord supplies the overlay safety root, Ω_N lifts resource namespaces into live owners, protocol morphisms preserve placement, and runtime effects remain typed. The implementation evidence supports those claims only within the declared finite scopes and reported measurements.

References

[1] N. Kshetri, "Privacy and security issues in cloud computing: The role of institutions and institutional evolution," Telecommunications Policy, vol. 37, no. 4–5, pp. 372–386, 2013.

[2] D. Zissis and D. Lekkas, "Addressing cloud computing security issues," Future Generation Computer Systems, vol. 28, no. 3, pp. 583–592, 2012.

[3] N. Kshetri and S. Murugesan, "Cloud computing and EU data privacy regulations," Computer, vol. 46, no. 3, pp. 86–89, 2013.

[4] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup service for internet applications," in Proceedings of the 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications. ACM, 2001, pp. 149–160.

[5] P. Zave, "Reasoning about identifier spaces: How to make Chord correct," IEEE Transactions on Software Engineering, vol. 43, no. 12, pp. 1144–1156, 2017, https://arxiv.org/abs/1610.01140.

[6] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in Stabilization, Safety, and Security of Distributed Systems, ser. Lecture Notes in Computer Science, vol. 6976. Springer, 2011, pp. 386–400.

[7] A. Shamir, "How to share a secret," Communications of the ACM, vol. 22, no. 11, pp. 612–613, 1979.

[8] B. Campbell, R. Mahy, and C. Jennings, "The message session relay protocol (MSRP)," RFC 4975, 2007, https://www.rfc-editor.org/info/rfc4975.

[9] L. Lamport, Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley, 2002.

[10] J. Nadal, "Stateright: Model checking for implementing distributed systems," https://www.stateright.rs/, accessed: July 4, 2026.

[11] D. Mazières, M. Kaminsky, M. F. Kaashoek, and E. Witchel, "Separating key management from file system security," in Proceedings of the 17th ACM Symposium on Operating Systems Principles, 1999, pp. 124–139.

[12] W3C Decentralized Identifier Working Group, "Decentralized identifiers (DIDs) v1.0: Core architecture, data model, and representations," W3C Recommendation, 2022, https://www.w3.org/TR/2022/REC-did-core-20220719/.

[13] B. W. Lampson, "Protection," ACM SIGOPS Operating Systems Review, vol. 8, no. 1, pp. 18–24, 1974.

[14] M. Abadi, M. Burrows, B. Lampson, and G. Plotkin, "A calculus for access control in distributed systems," ACM Transactions on Programming Languages and Systems, vol. 15, no. 4, pp. 706–734, 1993.

[15] E. Moggi, "Notions of computation and monads," Information and Computation, vol. 93, no. 1, pp. 55–92, 1991.

[16] G. Plotkin and M. Pretnar, "Handlers of algebraic effects," in Programming Languages and Systems, ser. Lecture Notes in Computer Science, vol. 5502, 2009, pp. 80–94.

[17] P. Maymounkov and D. Mazières, "Kademlia: A peer-to-peer information system based on the XOR metric," in Peer-to-Peer Systems, ser. Lecture Notes in Computer Science, vol. 2429. Springer, 2002, pp. 53–65.

[18] J. Benet, "IPFS - content addressed, versioned, P2P file system," arXiv:1407.3561 [cs.NI], 2014, https://arxiv.org/abs/1407.3561.

[19] Protocol Labs, "libp2p: A modular network stack," 2026, https://libp2p.io/.

[20] A. Rowstron and P. Druschel, "Pastry: Scalable, decentralized object location, and routing for large-scale peer-to-peer systems," in Middleware 2001, ser. Lecture Notes in Computer Science, vol. 2218, 2001, pp. 329–350.

[21] B. Y. Zhao, L. Huang, J. Stribling, S. C. Rhea, A. D. Joseph, and J. Kubiatowicz, "Tapestry: A resilient global-scale overlay for service deployment," IEEE Journal on Selected Areas in Communications, vol. 22, no. 1, pp. 41–53, 2004.

[22] S. Ratnasamy, P. Francis, M. Handley, R. Karp, and S. Shenker, "A scalable content-addressable network," in Proceedings of ACM SIGCOMM, 2001, pp. 161–172.

[23] E. Sit and R. Morris, "Security considerations for peer-to-peer distributed hash tables," in Peer-to-Peer Systems, ser. Lecture Notes in Computer Science, vol. 2429, 2002, pp. 261–269.

[24] M. Castro, P. Druschel, A. Ganesh, A. Rowstron, and D. S. Wallach, "Secure routing for structured peer-to-peer overlay networks," in 5th USENIX Symposium on Operating Systems Design and Implementation, 2002.

[25] J. R. Douceur, "The sybil attack," in Peer-to-Peer Systems, ser. Lecture Notes in Computer Science, vol. 2429, 2002, pp. 251–260.

[26] A. Singh, T.-W. J. Ngan, P. Druschel, and D. S. Wallach, "Eclipse attacks on overlay networks: Threats and defenses," in 25th IEEE International Conference on Computer Communications, 2006, pp. 1–12.

[27] H. Alvestrand, "Overview: Real-time protocols for browser-based applications," RFC 8825, 2021, https://www.rfc-editor.org/rfc/rfc8825.

[28] WebAssembly Working Group, "WebAssembly core specification," W3C Recommendation, 2019, https://www.w3.org/TR/2019/REC-wasm-core-1-20191205/.

[29] K. Nakagawa, M. Tsukada, K. Shima, and H. Esaki, "WebRTC-based measurement tool for peer-to-peer applications and preliminary findings with real users," arXiv:2112.02163, 2021, https://arxiv.org/abs/2112.02163.